

예비리포트

LAB2_FPGA 실험

제출일: 2026년 9월 21일

이준화 교수님

전자전기컴퓨터공학부

팀 B

학번: 2025440031

이름: 김영서

1. 실험 개요와 목적

1.1. 카운터, 클록 분주기, 레지스터, 시프트 레지스터, PISO 등의 회로를 설계하여 클록에 따른 데이터 저장과 상태 변화의 원리를 이해한다. 특히 순차회로에서는 현재 입력뿐만 아니라 이전 상태가 출력에 영향을 준다는 점과, 상승 에지를 기준으로 저장값이 갱신되는 과정을 파형을 통해 확인한다.

1.2. Moore FSM과 Mealy FSM을 포함한 여러 순차논리회로를 Verilog HDL로 구현하고, 테스트벤치를 이용해 입력과 클록 변화에 따른 상태 및 출력 변화를 검증한다. 정상 동작뿐 아니라 일부 RTL을 의도적으로 수정하여 자기검사 실패를 확인하고, 오류 시점의 입력과 현재 상태를 분석한 후 회로를 복구함으로써 순차논리 설계의 검증 방법을 익힌다.

1.3. VS Code에서 수행한 사전 시뮬레이션 결과를 바탕으로 Vivado에서 회로를 합성하고 FPGA에 구현한 뒤, 실제 보드의 스위치·버튼·LED·7-segment 등을 이용하여 설계한 순차논리회로의 동작을 확인한다. 이를 통해 RTL 시뮬레이션 결과와 실제 FPGA 하드웨어의 동작을 비교하고, 클록 입력과 상태 변화가 실제 장치에서 어떻게 나타나는지 확인하는 것을 목적으로 한다.

2. 실험의 이론적 배경

2.1. 순차논리회로와 클록

- ▶ 순차논리회로는 현재 입력뿐만 아니라 이전에 저장된 상태에 따라 출력이 결정되는 회로이다.
- ▶ 즉, 입력이 변한다고 즉시 저장값이 변경되는 것이 아니라, 정해진 클록 에지를 기준으로 상태가 갱신된다.
- ▶ LAB2의 테스트벤치에서는 클록 주기를 10 ns로 설정하며 상승 에지는 5, 15, 25, ... ns에서 발생한다.
- ▶ 입력이 변경되더라도 레지스터의 값은 다음 상승 에지에서 갱신되며, 테스트벤치는 nonblocking assignment에 의한 갱신이 완료된 뒤 값을 검사하기 위해 상승 에지 이후 1 ns를 기다린 후 결과를 비교한다.
- ▶ 또한 시뮬레이션에서 사용하는 10 ns 클록과 실제 FPGA 보드에서 사용하는 클록 조건은 서로 구분하여 해석해야 한다.

2.2. 동기식 Reset과 Enable

- ▶ 순차논리회로에서는 rst와 enable 등의 제어 신호를 이용해 상태 갱신을 제어한다.
- ▶ 본 실험에서 사용하는 동기식 reset은 rst=1인 순간 즉시 상태가 변하는 것이 아니라 다음 상승 에지에서 초기값으로 변경된다.
- ▶ 반면 rst=0, enable=1일 때는 회로가 정의한 다음 상태로 갱신되고, enable=0이면 기존 상태를 그대로 유지한다.
- ▶ 따라서 파형을 분석할 때는 reset 신호가 변하는 순간과 실제 상태가 바뀌는 상승 에지를 구분해야 하며, 각 회로에서 reset과 enable의 우선순위도 확인해야 한다.

2.3. Nonblocking Assignment와 저장값 갱신

- ▶ Verilog에서 순차논리의 저장값은 주로 nonblocking assignment인 `<=`를 이용하여 갱신한다.
- ▶ 여러 레지스터가 동일한 상승 에지에서 갱신되더라도 오른쪽 항은 에지 직전의 값을 기준으로 계산된다.
- ▶ 예를 들어 같은 상승 에지에서 A에 새로운 입력을 저장하고 B에 A를 전달하면, B에는 새로 갱신된 A가 아니라 상승 에지 직전의 A가 저장된다.
- ▶ 교안에서는 순차회로의 저장값에는 `<=`를 사용하고, 별도의 조합논리 `always @*` 블록에서는 `=`을 사용하도록 구분하고 있다.
- ▶ 이러한 특성은 Register, Shift Register, PISO, FSM 등의 파형을 해석할 때 중요하다.

2.4. 외부 버튼의 동기화, Debounce와 One-pulse

- ▶ FPGA 보드의 기계식 버튼은 한 번 눌러도 접점이 짧은 시간 동안 여러 번 변할 수 있는 바운스(bounce) 현상이 발생하며, FPGA 내부 클록과도 동기화되어 있지 않다.
- ▶ 따라서 교안에서는 외부 입력을 바로 상태 변경 신호로 사용하는 대신 동기화 => 안정 시간 확인 => 한 클록 펄스 생성의 순서로 처리한다.
- ▶ 두 단계의 동기화 플립플롭은 비동기 입력으로 인한 메타안정성이 내부 회로로 전달될 가능성을 낮추고, debounce 과정에서는 일정 시간 동안 입력이 안정적으로 유지되는지 확인하여 버튼의 반복적인 접점 변화를 제거한다.
- ▶ 마지막으로 실제 회로에는 한 번의 버튼 입력을 한 클록 폭의 펄스로 전달하여 한 번 눌렀을 때 상태가 여러 번 변하는 것을 방지한다.

2.5. Clock Divider와 Clock Enable

- ▶ 클록 분주기는 빠른 입력 클록을 이용하여 더 느린 동작 주기를 만드는 회로이다.
- ▶ 본 실험에서는 느린 동작을 LED나 파형으로 관찰하기 위해 분주 출력을 사용하며, 다른 순차회로에는 별도의 새로운 클록을 직접 공급하기보다 공통 clk를 유지하고 tick이 1인 상승 에지에서만 상태를 갱신하는 clock-enable 방식을 사용한다.
- ▶ 예를 들어 DIVISOR=10이면 tick은 10개의 클록마다 한 번 사용되며, 실제 클록 주파수와 분주 수의 비를 이용하여 최종 동작 속도를 계산할 수 있다.
- ▶ 클록 분주 실험에서는 단순히 주기만 확인하는 것이 아니라 첫 번째 출력 전환 시점과 duty 특성도 함께 확인한다.

2.6. Counter, Register, Shift Register와 PISO

- ▶ Up/Down Counter는 클록 에지마다 저장된 값을 증가 또는 감소시키는 순차회로이다.
- ▶ LAB2에서는 증가 시 $0 \Rightarrow \dots \Rightarrow 15 \Rightarrow 0$ 으로 순환하고, 0에서 감소하면 15가 되며, enable=0에서는 값을 유지한다.
- ▶ 또한 reset과 enable이 동시에 활성화된 경우 reset이 우선하도록 설계한다.
- ▶ Register는 입력 데이터를 클록 에지에 맞추어 저장하는 회로이다.
- ▶ 본 실험의 Register 회로에서는 load가 활성화되면 입력값을 stored에 저장하고, transfer가 활성화되면 저장된 값을 다른 레지스터로 전달한다.
- ▶ load와 transfer가 동시에 활성화될 경우 nonblocking assignment의 특성에 의해 전달받는 값은 같은 에지에서 새로 저장되는 값이 아니라 이전 stored 값이다.
- ▶ Shift Register는 클록 에지마다 저장된 여러 비트의 위치를 한 칸씩 이동시키는 회로이다.

- ▶ 본 실험에서는 새로운 직렬 입력이 bit 3으로 들어오고 기존 데이터는 bit 0 방향으로 이동한다.
- ▶ 따라서 연속된 입력에 따라 내부 저장값이 단계적으로 변화하는 모습을 파형에서 확인할 수 있다.
- ▶ PISO(Parallel-In Serial-Out)는 여러 비트의 데이터를 한 번에 병렬로 저장한 뒤 한 비트씩 직렬로 출력하는 회로이다.
- ▶ 예를 들어 1010을 병렬 로드하면 직렬 출력은 최상위 비트부터 $1 \rightarrow 0 \rightarrow 1 \rightarrow 0$ 순서로 나타난다.
- ▶ 또한 load와 enable이 동시에 활성화된 경우에는 load 동작이 우선한다.

2.7. FSM과 Moore/Mealy 상태 머신

- ▶ FSM(Finite State Machine)은 현재 상태와 입력에 따라 다음 상태를 결정하며 정해진 상태 사이를 전이하는 순차논리회로이다.
- ▶ Moore FSM에서는 출력이 현재 상태에 의해 결정된다. LAB2의 Moore 회로는 $00 \Rightarrow 01 \Rightarrow 10 \Rightarrow 00$ 의 순서로 상태가 전이하며, enable과 advance가 모두 활성화된 유효 에지에서만 다음 상태로 이동한다.
- ▶ 출력은 상태 자체에 의해 결정되므로 클록 사이에서 advance 입력만 변경하더라도 출력은 즉시 바뀌지 않는다.
- ▶ 반면 Mealy FSM에서는 출력이 현재 상태와 현재 입력에 모두 의존한다.
- ▶ 따라서 상태가 바뀌지 않더라도 입력이 바뀌면 조합 출력은 변경될 수 있다.
- ▶ 본 실험에서는 입력이 1인 유효 에지에서 $S0 \leftrightarrow S1$ 로 상태가 전이하고 입력이 0이면 상태를 유지한다.
- ▶ 또한 enable=0은 상태 갱신만 정지시키므로 입력 변화에 따른 조합 출력의 변화는 계속 발생할 수 있다.

2.8. 7-Segment Dynamic Scanning

- ▶ 여러 자리의 7-segment를 제한된 신호선으로 구동하기 위해 자리 스캔 방식을 사용한다.
- ▶ LAB2에서는 8자리 중 한 자리만 순간적으로 활성화하고 이를 빠르게 순환하여 사람의 눈에는 여러 자리가 동시에 켜져 있는 것처럼 보이게 한다.
- ▶ 자리와 자리 사이에는 모든 자리 선택 신호를 비활성화하는 blank 구간을 두어 다음 자리의 segment 데이터가 안정될 시간을 확보한다.
- ▶ 또한 기능 코어에서는 active-high 형태의 논리값을 생성하고, 실제 보드에서 필요한 자리 선택 및 segment의 극성 변환은 핀 연결 부분에서 처리한다.

3. 실험에 필요한 장치

3.1. Visual Studio Code

- 본 실험에서 Verilog HDL 파일과 SystemVerilog Testbench를 작성하는 주된 편집 환경이다.
- src 폴더의 각 조합논리회로, 통합 모듈, 버튼 및 LCD 제어 모듈을 작성하고, sim 폴더의 Testbench와 FPGA 핀 설정을 위한 XDC 파일도 관리한다.
- 즉, 회로를 HDL이라는 소프트웨어 형태로 기술하는 첫 단계에 사용된다.

3.2. Icarus Verilog

- HDL로 작성한 회로를 실제 FPGA에 구현하기 전에 기능이 올바른지 확인하는 시뮬레이터이다.
- 이번 실험에서는 10개 mode와 각각의 256개 DIP 입력 조합을 검사하여 총 2560개의 경우를 자동으로 비교한다.
- 또한 reset, button bounce, long press, MODE 10에서 MODE 01로의 순환 및 LCD 동작도 Testbench에서 함께 검증한다.

3.3. VaporView

- Icarus Verilog 시뮬레이션에서 생성된 VCD 파일을 파형으로 표시하는 도구이다.
- 본 실험에서는 여러 신호를 관찰하여 버튼 입력 이후 모드가 바뀌는 과정과 LCD 데이터가 전송되는 과정을 시간축에서 확인할 예정이다.

3.4. AMD Vivado 2026.1

- Vivado는 Verilog로 기술한 논리회로를 실제 AMD/Xilinx FPGA에서 사용할 수 있는 회로로 변환하는 FPGA 설계 프로그램이다.
- 사전 시뮬레이션 이후 회로를 합성하고, FPGA 내부 자원에 회로를 배치하고 연결하는 과정을 수행하며, 마지막으로 FPGA에 기록할 비트 스트림을 생성한다.

****FPGA와 이러한 소프트웨어 도구의 관계****

- FPGA는 ASIC과 달리 반도체가 제조된 이후에도 논리 기능을 다시 설정할 수 있는 소자이다.
- ASIC은 특정 기능에 맞는 회로를 실제 반도체 제조 공정을 통해 고정하여 제작하므로, 설계에 문제가 발생하면 경우에 따라 설계를 수정한 뒤 다시 제작해야 한다.
- 반면, FPGA는 이미 제조된 칩의 programmable logic과 interconnect를 Bitstream으로 설정하기 때문에 새로운 반도체 제조 공정을 반복하지 않고도 회로를 변경할 수 있다.
- 이러한 위험성 측면, 시간적 측면에 따라 반도체 제조 공정에서는 FPGA를 사용한다.

4. 모드별 입출력 표

회로	주요 입력	실행 조건	주요 출력	예상 동작
Up/Down Counter	SW8 = sw[0]	N4 1회	LED[3:0] = count	SW8=0이면 +1, SW8=1이면 -1
Clock Divider	별도 스위치 입력 없음	자동 동작	LED0 = / 2 , LED1 = / 10 , LED2 = / 50 , LED3 = / 1000 , LED4=tick	1 kHz 입력 클록을 여러 비율로 분주
Register Pair	SW1~4 = sw[7:4] data, SW7=sw[1] transfer, SW8=sw[0] load	N4 1회	LED[7:4]=stored, LED[3:0]=registered(value)	입력 저장 및 저장값 전달
Shift Register	SW1 = sw[7] serial input	N4 1회	LED[3:0]=shifted value	직렬 입력을 넣으며 4비트 저장값 이동
PISO	SW1~4=sw[7:4] 병렬 데이터, SW8=sw[0] load/shift 선택	N4 1회	LED[7:4]=parallel 저장값, LED0=serial_out	SW8=1이면 load, SW8=0이면 shift
Moore FSM	SW1=sw[7] advance	N4 1회	LED[1:0]=state/output	advance = 1 에서 00=>01=>10=>00
Mealy FSM	SW1=sw[7] input	N4로 상태 전이	LED2 = state , LED[1:0]=output	출력이 현재 상태와 입력에 모두 의존
8 - Digit Segment Scan	SW1~4=sw[7:4] 첫 자리 데이터	자동 동작, N4 불필요	7 - segment seg_com , seg_data , LED[2:0]=scan_index	8개 자리를 순차적으로 자동 스캔

5. 각 verilog 파일의 역할

파일명	역할
counter4.v	4비트 Up/Down Counter를 구현한다. enable이 활성화된 클록 에지에서 설정된 방향에 따라 값을 1씩 증가 또는 감소시키고, reset 시 0으로 초기화한다.
clock_divider.v	입력 클록을 설정된 분주비에 따라 낮은 주파수의 출력 및 tick 신호로 변환한다. 통합 회로에서는 여러 속도의 분주 신호를 관찰하는 데 사용한다.
register_pair.v	두 개의 4비트 레지스터를 이용하여 입력 데이터를 저장하고, 저장된 값을 다른 레지스터로 전달하는 동작을 수행한다.
shift_register4.v	4비트 시프트 레지스터를 구현한다. 한 번의 유효 클록마다 새로운 직렬 입력을 저장하면서 기존 비트들을 한 방향으로 이동시킨다.
piso4.v	4비트 PISO(Parallel-In Serial-Out) 레지스터를 구현한다. 병렬 입력을 한 번에 저장한 뒤 저장된 데이터를 한 비트씩 직렬로 출력한다.
moore_cycle.v	Moore 방식의 FSM을 구현한다. 입력 조건과 유효 클록에 따라 상태가 순차적으로 전이하며, 출력은 현재 상태에 의해 결정된다.
mealy_toggle.v	Mealy 방식의 FSM을 구현한다. 상태 전이를 수행하며 출력은 현재 상태와 현재 입력에 의해 결정된다.
segment_scan8.v	8자리 7-segment를 동적 스캔 방식으로 구동한다. 한 번에 한 자리씩 선택하고 자리 사이에 blank 구간을 두면서 8자리를 반복적으로 표시한다.
input_frontend.v	외부 버튼 및 스위치 입력을 FPGA 내부에서 안정적으로 사용할 수 있도록 입력 동기화, 안정 상태 확인 등의 전처리를 수행한다.
button_onepulse.v	버튼 입력을 처리하여 한 번의 버튼 조작마다 한 클록 폭의 펄스를 생성한다. 버튼을 길게 눌러도 한 번의 동작만 발생하도록 한다.
lcd_lab2_modes.v	현재 LAB2 모드 번호와 각 회로의 이름을 LCD에 표시하기 위한 LCD 제어 신호와 데이터를 생성한다.
lab2_integrated.v	LAB2 통합 회로의 최상위 모듈이다. 각 기능 코어를 인스턴스화하고 mode에 따라 해당 회로를 활성화하며, 버튼·스위치 입력과 LED·LCD·7-segment 출력을 연결하고 선택한다.

6. 사전 시뮬레이션 결과

6.1. Counter

작성한 코드는 다음과 같다.

```
src > ≡ counter4.v
1  `timescale 1ns/1ps
2  module counter4 (
3      input wire clk, rst, enable, down,
4      output reg [3:0] value
5  );
6
7      always @(posedge clk) begin
8          if (rst) value <= 4'd0;
9          else if (enable) begin
10             if (down) value <= value - 4'd1;
11             else value <= value + 4'd1;
12         end
13     end
14 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

Problems Output Debug Console **Terminal** Ports

Python 3.14.7

RUN "C:\Program Files\Git\cmd\git.EXE" --version
git version 2.55.0.windows.5

RUN C:\msys64\mingw64\bin\iverilog.EXE -V ...

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.

* Terminal will be reused by tasks, press any key to close it.

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
● RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\lab2_01_counter\src -I C:\Users\young\Documents\lab2_01_counter\sim -s tb_counter4 -o C:\Users\young\Documents\lab2_01_counter\build\sim\run-22e81847d9a045f78e5b1e34c89b3075\sim.vvp C:\Users\young\Documents\lab2_01_counter\src\counter4.v C:\Users\young\Documents\lab2_01_counter\src\input_frontend.v C:\Users\young\Documents\lab2_01_counter\src\lab2_counter.v C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\lab2_01_counter\build\sim\run-22e81847d9a045f78e5b1e34c89b3075\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS counter4 checks=36
C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv:40: $finish called at 356000 (1ps)

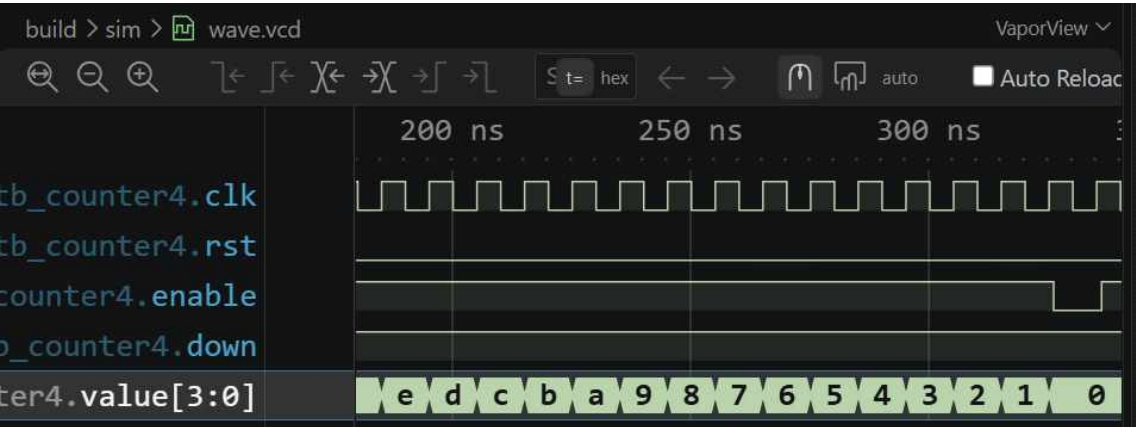
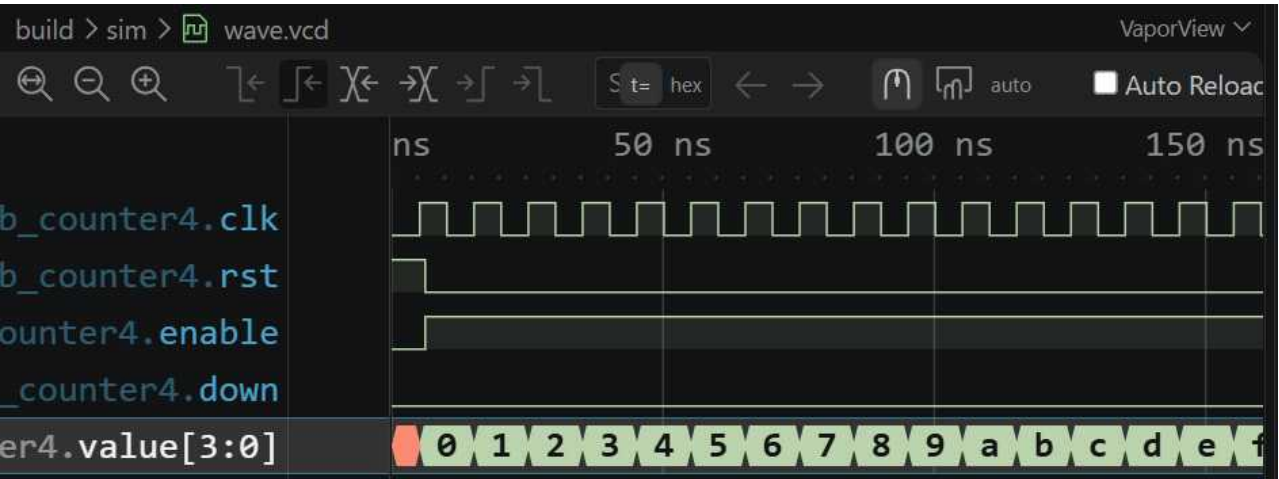
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\lab2_01_counter\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

위 터미널 응답에서 알 수 있듯이 36개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 356ns로 올바르게 출력되었다.

이후 파형 출력을 통해 코드가 정상 작동하는지
확인하였다.

구간	rst	enable	down	value 예상
초기 Reset	1	0	0	0
Up Count	0	1	0	0 → 1 → 2 → ... → 15 → 0
Down Count	0	1	1	0 → 15 → 14 → ... → 1 → 0
Hold	0	0	1	0 유지
Down Wrap	0	1	1	0 → 15
마지막 Reset	1	1	1	15 → 0

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
always @(posedge clk) begin
    if (rst) value <= 4'd0;
    else if (enable) begin
        if (down) value <= value - 4'd1;
        else value <= value + 4'd1;
    end
end
```

+부분을 -로 바꾸어 실행하였다.

시뮬레이션 결과 오류가 발생하였다.

```
④ RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\lab2_01_counter\src -I C:\Users\young\Documents\lab2_01_counter\sim -s tb_counter4 -o C:\Users\young\Documents\lab2_01_counter\build\sim\run-5279cf15958c4ba3aaa8db0220a163d5\sim.vvp C:\Users\young\Documents\lab2_01_counter\src\counter4.v C:\Users\young\Documents\lab2_01_counter\src\input_frontend.v C:\Users\young\Documents\lab2_01_counter\src\lab2_counter.v C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\lab2_01_counter\build\sim\run-5279cf15958c4ba3aaa8db0220a163d5\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL up including 15 to 0 time=16000
FATAL: C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv:29: check failed
      Time: 16000  Scope: tb_counter4.check

ERROR: 종료 코드 1: C:\Users\young\Documents\lab2_01_counter\build\sim\run-5279cf15958c4ba3aaa8db0220a163d5\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
else if (enable) begin
    if (down) value <= value - 4'd1;
    else value <= value + 4'd1;
end
```

```
● RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\lab2_01_counter\src -I C:\Users\young\Documents\lab2_01_counter\sim -s tb_counter4 -o C:\Users\young\Documents\lab2_01_counter\build\sim\run-859ea3d7ffdb423794e35fa1906ff79a\sim.vvp C:\Users\young\Documents\lab2_01_counter\src\counter4.v C:\Users\young\Documents\lab2_01_counter\src\input_frontend.v C:\Users\young\Documents\lab2_01_counter\src\lab2_counter.v C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\lab2_01_counter\build\sim\run-859ea3d7ffdb423794e35fa1906ff79a\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS counter4 checks=36
```

```
C:\Users\young\Documents\lab2_01_counter\sim\tb_counter4.sv:40: $finish called at 356000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\lab2_01_counter\build\sim\wave.vcd
```

```
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 즉, 원래는 down이 0일 때 0부터 시작하여 1씩 커져야 하지만, 0에서 시작하여 F, E, D처럼 감소하게 되어 기댓값과 다른 출력을 얻어 오류가 발생한 것이다.

6.2. Clock Divider

작성한 코드는 다음과 같다.

```
circuits > 02_clock_divider > src > ≡ clock_divider.v
1  `timescale 1ns/1ps
2  // DIVISOR must be an even integer >= 2.
3  // divided is an observation output; use tick as an enable on clk
4  module clock_divider #(parameter integer DIVISOR = 10) (
5      input wire clk, rst,
6      output reg divided,
7      output wire tick
8  );
9      localparam integer WIDTH = $clog2(DIVISOR);
10     reg [WIDTH-1:0] count;
11     assign tick = (!rst && (count == DIVISOR - 1));
12     always @(posedge clk) begin
13         if (rst) begin
14             count <= 0;
15             divided <= 1'b0;
16         end else begin
17             if (count == DIVISOR - 1) count <= 0;
18             else count <= count + 1'b1;
19             if (count == DIVISOR/2 - 1) divided <= 1'b1;
20             else if (count == DIVISOR - 1) divided <= 1'b0;
21         end
22     end
23 endmodule
24
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
* Terminal will be reused by tasks, press any key to close it.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim -s tb_clock_divider -o C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-1e1cf1721ebd4090844771d52a3c6caf\sim.vvp
p C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\input_frontend.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\lab2_clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sv
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-1e1cf1721ebd4090844771d52a3c6caf\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS clock_divider checks=129
```

```
C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sv:19: $finish called at 436000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\wave.vcd
```

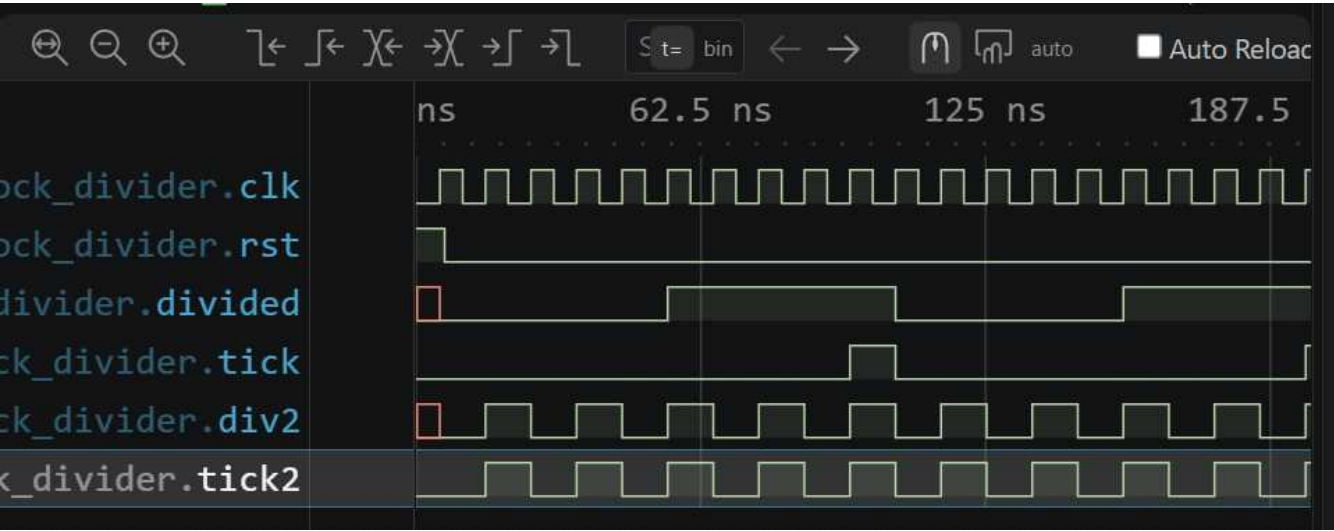
위 터미널 응답에서 알 수 있듯이 129개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

또한 출력 시간 또한 436ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

clk 상승엿지	count 변화 후	divided	tick	의미
15 ns	1	0	0	Low 구간
25 ns	2	0	0	Low
35 ns	3	0	0	Low
45 ns	4	0	0	Low
55 ns	5	1	0	divided 상승
65 ns	6	1	0	High
75 ns	7	1	0	High
85 ns	8	1	0	High
95 ns	9	1	1	tick 발생
105 ns	0	0	0	다음 주기 시작

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
end else begin
    if (count == DIVISOR - 1) count <= 0;
    else count <= count + 1'b1;
    if (count == DIVISOR/2) divided <= 1'b1;
    else if (count == DIVISOR - 1) divided <= 1'
```

코드 중 -1 부분을 제거하여 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
④ RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src -I C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim -s tb_clock_divider -o C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-00db54cffd2544f89edcdb4bd9728ffc\sim.vvp C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\input_frontend.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\src\lab2_clock_divider.v C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sim.vv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-00db54cffd2544f89edcdb4bd9728ffc\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL minimum divisor duty time=16000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sim:12: check failed
      Time: 16000 Scope: tb_clock_divider.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-00db54cffd2544f89edcdb4bd9728ffc\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
        divided <= 1'b0;
    end else begin
        if (count == DIVISOR - 1) count <= 0;
        else count <= count + 1'b1;
        if (count == DIVISOR/2 - 1) divided <= 1'b1;
        else if (count == DIVISOR - 1) divided <= 1'b0;
    end
end
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\run-ded6bf3d496d41b9a08cbdf03cf48156\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS clock_divider checks=129
```

```
C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\sim\tb_clock_divider.sv:19: $finish called at 436000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\fpga_lab2\circuits\02_clock_divider\build\sim\wave.vcd
```

```
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 이렇게 바꾸면 divided의 상승 시점이 원래보다 한 클록 늦어져, 특히 DIVISOR=2 인스턴스의 듀티 동작이 TB 예상값과 어긋나게 되어 오차가 발생하게 된 것이다.

6.3. Register

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2  module register_pair (
3      input wire clk, rst, load, transfer,
4      input wire [3:0] data_in,
5      output reg [3:0] stored, value
6  );
7      always @(posedge clk) begin
8          if (rst) begin
9              stored <= 4'd0;
10             value <= 4'd0;
11         end else begin
12             if (load) stored <= data_in;
13             if (transfer) value <= stored;
14         end
15     end
16 endmodule
17
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
* Terminal will be reused by tasks, press any key to close it.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-ed2c12b68bec4dae8147c870b3093d4b\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim\tb_register_pair.sv:19: $finish called at 66000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

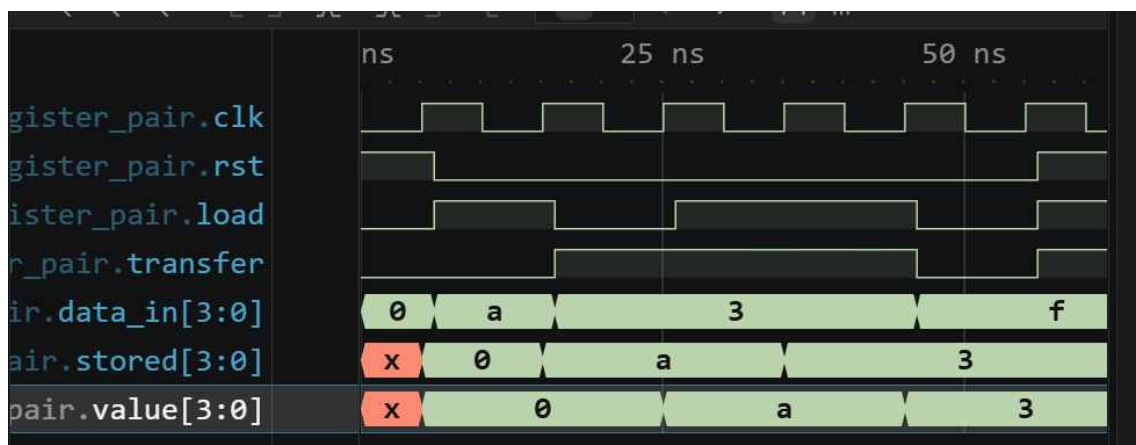
위 터미널 응답에서 알 수 있듯이 7개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

또한 출력 시간 또한 66ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

검사 시각	동작	stored	value
6 ns	reset	0	0
16 ns	A 저장	A	0
26 ns	입력은 3, 기존 저장값 전달	A	A
36 ns	3 저장 + 전달 동시	3	A
46 ns	다음 edge에도 동시	3	3
56 ns	둘 다 0, 입력 F	3	3
66 ns	reset 우선	0	0

위의 표의 결과가 시뮬레이션의 예상 결과값이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
11      end else begin
12          if (load) stored <= data_in;
13          if (transfer) value <= data in;
14      end
15  end
```

코드의 'stored' 부분을 'data in'으로 바꾸어 오류를 발생시키겠다.

시뮬레이션 결과 오류가 발생하였다.

```
⊙ RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documents\fpga_lab2\circuits\03_register\src -I C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim -s tb_register_pair -o C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-7c150e77be854fa8bf6d92f4f72dcdf2\sim.vvp C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\register_pair.v C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\input_frontend.v C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\lab2_register.v C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim\tb_register_pair.sv
C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\register_pair.v:13: syntax error
C:\Users\young\Documents\fpga_lab2\circuits\03_register\src\register_pair.v:13: error: Malformed statement

ERROR: 종료 코드 2: C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-7c150e77be854fa8bf6d92f4f72dcdf2\compile.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이션한 결과 복구에 성공하였다.

```
end else begin
    if (load) stored <= data_in;
    if (transfer) value <= stored;
end
end
```

```
03_register\sim\tb_register_pair.v
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\run-70b9d5280a9e41d7b1994d424ce2f316\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\young\Documents\fpga_lab2\circuits\03_register\sim\tb_register_pair.v:19: $finish called at 66000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\03_register\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 위에서와 같이 코드를 변경하게 되면, 저장된 값을 전달하는 것이 아닌 현재 입력을 바로 전달하게 된다. 이에 따라 register의 원래 동작과 다르게 되어 오류가 발생한 것이다.

6.4. Shift register

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2
3  module shift_register4 (
4      input wire clk, rst, enable, serial_in,
5      output reg [3:0] value
6  );
7      always @(posedge clk) begin
8          if (rst) value <= 4'd0;
9          else if (enable) value <= {serial_in, value[3:1]};
10     end
11 endmodule
12
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-ad36d0b652524646b-aa6d5ca10cf280b\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS shift_register4 checks=8
C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\sim\tb_shift_register4.sv:30: $finish called at 106000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\wave.vcd

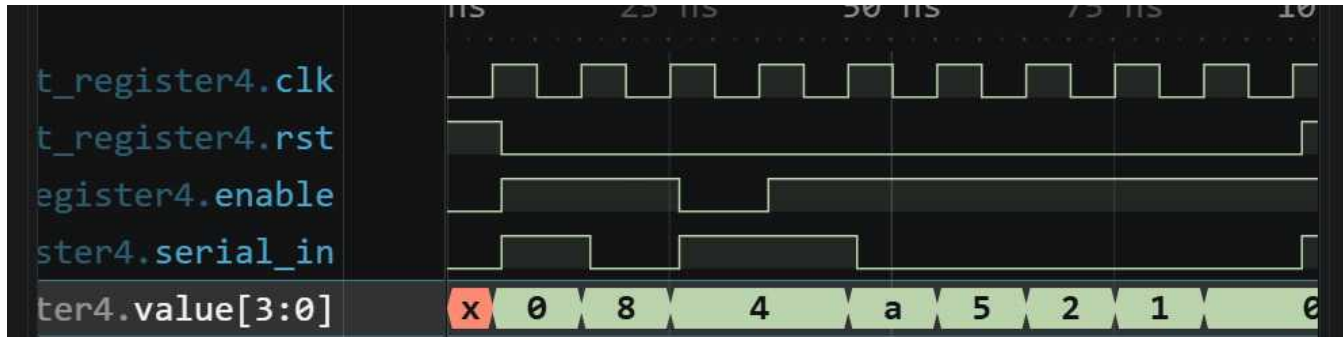
```

위 터미널 응답에서 알 수 있듯이 8개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 106ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

검사 시각	동작	value
6 ns	reset	0000
16 ns	serial_in=1, enable=1	1000 = 8
26 ns	serial_in=0	0100 = 4
36 ns	enable=0, 입력은 1	0100 유지
46 ns	enable=1, 입력 1	1010 = A
56 ns	입력 0	0101 = 5
96 ns	0을 네 번 더 입력	0000
106 ns	rst=1	0000

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
8         if (rst) value <= 4'd0;  
9         else if (enable) value <= {value[2:0], value[3:1]};  
0     end  
1 endmodule
```

코드의 부등호 'serial_in' 을 'value[2:0]' 으로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-809f180ef10843a79f830aaec7916e3\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL shift 1 time=16000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\sim\tb_shift_register4.sv:22: check failed
      Time: 16000 Scope: tb_shift_register4.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-809f180ef10843a79f830aaec7916e3\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
8      if (rst) value <= 4'd0;
9      else if (enable) value <= {serial_in, value[3:1]};
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\run-4462d92ff2e640aaae2f84404adb38f5\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS shift_register4 checks=8
C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\sim\tb_shift_register4.sv:30: $finish called at 106000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\04_shift_register\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.

- 의도적으로 코드를 바꾸게 되면서 시프트 방향을 반대로 바꾸었다. 이에 따라 예측값과 다른 출력값을 얻어 오차가 발생한 것이다.

6.5. PISO

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2
3  module piso4 (
4      input wire clk, rst, load, enable,
5      input wire [3:0] data_in,
6      output wire serial_out,
7      output reg [3:0] value
8  );
9      assign serial_out = value[3];
10
11     always @(posedge clk) begin
12         if (rst) value <= 4'd0;
13         else if (load) value <= data_in;
14         else if (enable) value <= {value[2:0], 1'b0};
15     end
16 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-49736e3815e642d59838167928320f74\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS piso4 checks=114
C:\Users\young\Documents\fpga_lab2\circuits\05_piso\sim\tb_piso4.sv:30: $finish called at 976000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\wave.vcd
```

위 터미널 응답에서 알 수 있듯이 114개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 976ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

이론에 따르면,

만약 입력이 A=1010인 구간은

616 ns → A 병렬 load 확인

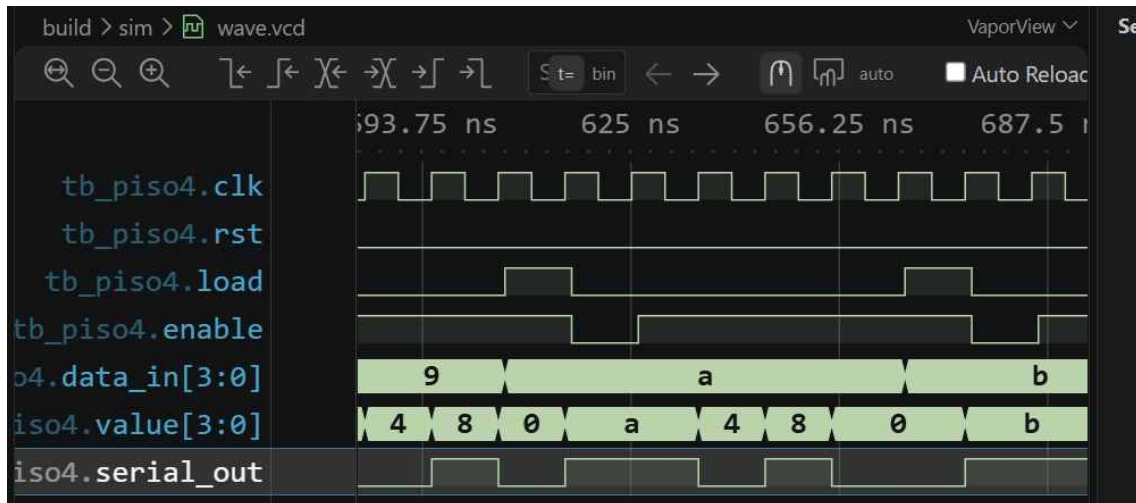
626 ns → hold, serial_out = 1

636 ns → serial_out = 0

646 ns → serial_out = 1

656 ns → serial_out = 0

666 ns → 네 번 shift 후 value = 0000 으로 출력되어야 한다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```

8  );
9  assign serial_out = value[0];
10
11 always @(posedge clk) begin

```

코드의 value값을 3에서 0으로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-7b5ea152f2094b8fa3734faf7db8ee0d\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL MSB first before edge time=86000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\05_piso\sim\tb_piso4.sv:22: check failed
      Time: 86000  Scope: tb_piso4.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-7b5ea152f2094b8fa3734faf7db8ee0d\simulation.log
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
9      assign serial_out = value[3];
10
11      always @(posedge clk) begin
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\05_piso\build\sim\run-9a8e9aa46abe420294069f5106d55526\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS piso4 checks=114
C:\Users\young\Documents\fpga_lab2\circuits\05_piso\sim\tb_piso4.sv:30: $finish called at 976000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- value값을 의도적으로 3에서 0으로 바꾸면서 MSB가 아닌 LSB를 출력하도록 하였다. 이에 따라 오류가 발생한 것이다.

6.6. Moore

작성한 코드는 다음과 같다.

```
circuits > 06_moore > src > ≡ moore_cycle.v
1  `timescale 1ns/1ps
2
3  module moore_cycle (
4      input wire clk, rst, enable, advance,
5      output reg [1:0] value
6  );
7      always @(posedge clk) begin
8          if (rst) value <= 2'b00;
9          else if (enable && advance) begin
10             case (value)
11                 2'b00: value <= 2'b01;
12                 2'b01: value <= 2'b10;
13                 default: value <= 2'b00;
14             endcase
15         end
16     end
17 endmodule
18
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or
modify it under the terms of the GNU General Public License as published
by the Free Software Foundation; either version 2 of the License,
or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program; if not, write to the Free Software Foundation,
Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성
하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로
다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\
06_moore\build\sim\run-17a4eb22e4cf4df3b832f6294466e5e3\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS moore_cycle checks=23
C:\Users\young\Documents\fpga_lab2\circuits\06_moore\sim\tb_moore_cycle.sv:30: $f
inish called at 226000 (1ps)

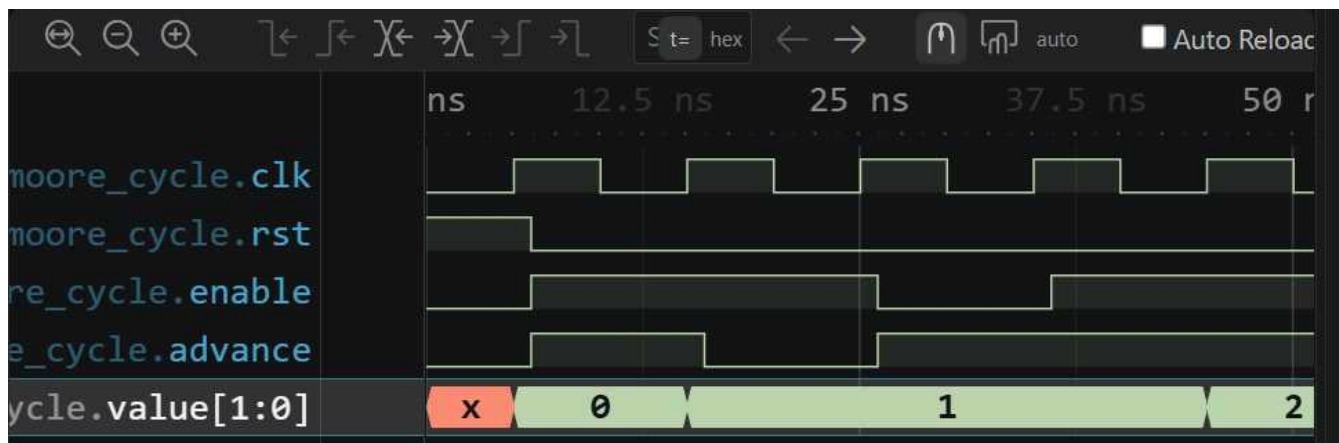
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\wave.vcd
```

위 터미널 응답에서 알 수 있듯이 23개의 경우 모두
올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 226ns로 올바르게 출력되었다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

이론에 따르면 첫 번째 상태 순환은 다음과 같아야 한다.

6 ns : reset → 00
7 ns : advance=1 → 여전히 00
16 ns : 유효 edge → 01
26 ns : advance=0 → 01 유지
36 ns : enable=0 → 01 유지
46 ns : 둘 다 1 → 10
56 ns : 다음 유효 edge → 00



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```
10      case (value)
11      |      2'b00: value <= 2'b01;
12      |      2'b01: value <= 2'b00;
13      |      default: value <= 2'b00;
```

코드의 '10' 을 '00' 으로 바꾸어 오류를 발생시켰다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\run-27cee18768454f44a26da51602fb109b\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL S1 to S2 time=46000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\06_moore\sim\tb_moore_cycle.sv:22: check failed
      Time: 46000  Scope: tb_moore_cycle.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\run-27cee18768454f44a26da51602fb109b\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
```

시뮬레이션 결과 오류가 발생하였다.

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
10      case (value)
11      |    2'b00: value <= 2'b01;
12      |    2'b01: value <= 2'b10;
13      |    default: value <= 2'b00;
14      endcase
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\run-a37d16003da54e87947ea3643873b8aa\sim.vvp
```

```
VCD info: dumpfile wave.vcd opened for output.
```

```
LAB2_PASS moore_cycle checks=23
```

```
C:\Users\young\Documents\fpga_lab2\circuits\06_moore\sim\tb_moore_cycle.sv:30: $finish called at 226000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

```
WAVE C:\Users\young\Documents\fpga_lab2\circuits\06_moore\build\sim\wave.vcd
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 의도적으로 코드를 변형하여 상태가 s1에서 s2로 넘어가야 하는 상황에서 s0으로 넘어가게 되어 오류가 발생한 것이다.

6.7. Mealy

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2
3  module mealy_toggle (
4      input wire clk, rst, enable, bit_in,
5      output reg state,
6      output wire [1:0] value
7  );
8      always @(posedge clk) begin
9          if (rst) state <= 1'b0;
10         else if (enable && bit_in) state <= ~state;
11     end
12
13     // Reset changes state at the rising edge; value is
14     assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);
15 endmodule
16
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\run-967cc398478c44fcb3cd4ba63f90773d\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS mealy_toggle checks=11
C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\sim\tb_mealy_toggle.sv:30: $finish called at 67000 (1ps)
```

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

위 터미널 응답에서 알 수 있듯이 11개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 67ns로 올바르게 출력되었다.

이론에 따르면 각 상태별 결과는 다음과 같이 얻어져야한다.

state = 0

bit_in = 0

value = 00

일 때, bit_in = 1이 되면,

state = 0

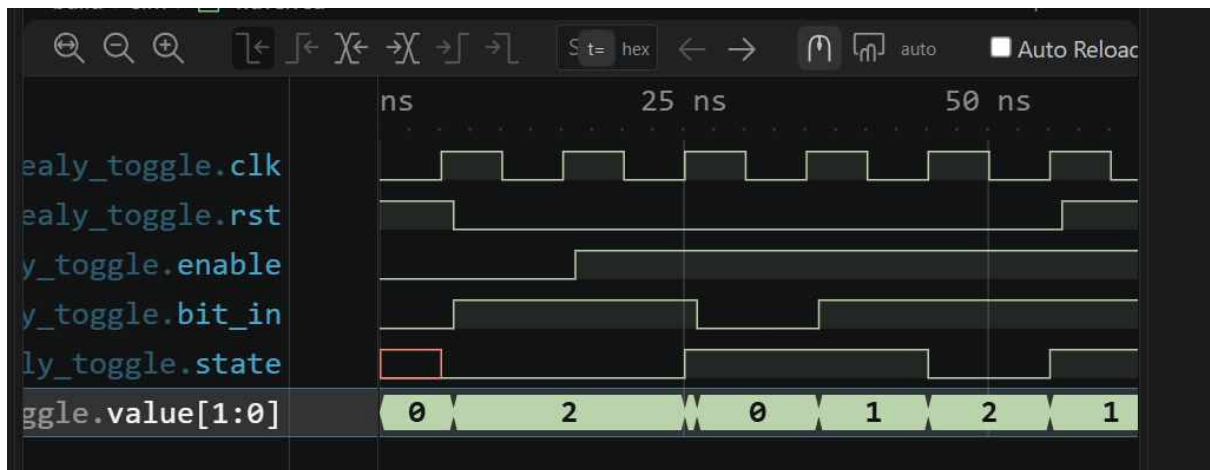
value = 10 으로 바뀐다.

반대로 state = 1이면,

bit_in = 1 => value = 01

bit_in = 0 => value = 00

로 변화한다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```

11     end
12
13     // Reset changes state at the rising edge; value is combinational
14     assign value = !bit_in ? 2'b00 : (state ? 2'b10 : 2'b01);
15 endmodule
16

```

코드의 부등호 '01' 과 '10' 을 서로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\run-4657059703f64c3b945d39a3fc7f1f81\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL S0 input changes between clocks time=7000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\sim\tb_mealy_toggle.sv:
22: check failed
      Time: 7000  Scope: tb_mealy_toggle.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\
run-4657059703f64c3b945d39a3fc7f1f81\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft\WindowsApps\python
.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
11     end
12
13     // Reset changes state at the rising edge; value is combinational
14     assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);
15 endmodule
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\run-fad8ce1e69e043ee9d295ab0f92503ed\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS mealy_toggle checks=11
C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\sim\tb_mealy_toggle.sv:30: $finish called at 67000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\07_mealy\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 정상은 state=0, bit_in=1 => 10, state=1, bit_in=1 => 01 이지만, 오류 회로에서는 state=0, bit_in=1 => 01 , state=1, bit_in=1 => 10 으로 바뀌게 되므로 오류가 발생한 것이다.

6.8. Segment Scan

작성한 코드는 다음과 같다.

```
1  `timescale 1ns/1ps
2  // Logical outputs are active-high. Apply board polarity in the wrapper.
3  // Each slot contains one active interval followed by one blank interval.
4  module segment_scan8 (
5      input wire clk, rst, enable,
6      input wire [31:0] digits,
7      output reg [7:0] select,
8      output reg [7:0] segments,
9      output reg [2:0] index
10 );
11     reg blank;
12     reg [3:0] nibble;
13
14     always @(posedge clk) begin
15         if (rst) begin
16             index <= 0;
17             blank <= 1'b1;
18         end else if (enable) begin
19             blank <= ~blank;
20             if (!blank)
21                 index <= index + 1'b1;
22         end
23     end
24
25     always @* begin
26         nibble = digits >> (index * 4);
27         select = blank ? 8'h00 : (8'h01 << index);
28
29         // abcdefg, dp; decimal point remains off.
30         case (nibble)
31             4'h0: segments = 8'b11111100;
32             4'h1: segments = 8'b01100000;
33             4'h2: segments = 8'b11011010;
34             4'h3: segments = 8'b11110010;
35             4'h4: segments = 8'b01100110;
36             4'h5: segments = 8'b10110110;
37             4'h6: segments = 8'b10111110;
38             4'h7: segments = 8'b11100000;
39             4'h8: segments = 8'b11111110;
40             4'h9: segments = 8'b11110110;
41             4'ha: segments = 8'b11101110;
42             4'hb: segments = 8'b00111110;
43             4'hc: segments = 8'b10011100;
44             4'hd: segments = 8'b01111010;
45             4'he: segments = 8'b10011110;
46             4'hf: segments = 8'b10001110;
47             default: segments = 8'h00;
48         endcase
49     end
50 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -V
Icarus Verilog runtime version 13.0 (stable) (v13_0)

Copyright (c) 2001-2026 Stephen Williams (steve@icarus.com)

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License along
with this program; if not, write to the Free Software Foundation, Inc.,
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.

도구 확인 완료. 다음은 src/.sim/.constraints/의 파일을 직접 작성하세요.
```

check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-2ff706ec968648809f206456b6d23a57\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS segment_scan8 checks=194
C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\sim\tb_segment_scan8.sv:30: $finish called at 1306000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\wave.vcd
* Terminal will be reused by tasks, press any key to close it.
```

위 터미널 응답에서 알 수 있듯이 194개의 경우 모두
올바르게 출력되었다는 것을 알 수 있다.
또한 출력 시간 또한 1306ns로 올바르게 출력되었다.

이론에 따르면 다음과 같은 결과가 얻어져야한다.

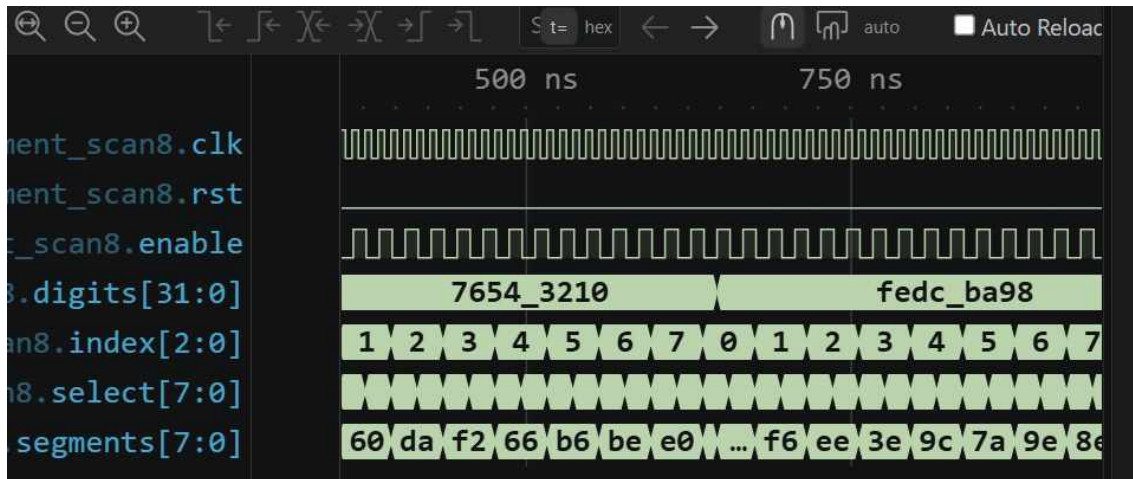
자리 활성화

=> enable=0이라도 활성화 상태 유지

=> blank

=> enable=0이라도 blank 유지

=> 다음 자리



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

```

24
25     always @* begin
26         nibble = digits >> (index * 4);
27         select = (8'h01 << index);
28
29         // abcdefg, dp; decimal point remains off.
30         case (nibble)

```

코드의 'select = blank ? 8'h00 : (8'h01 << index);' 을 'select = (8'h01 << index);' 으로 바꾸어 오류를 발생시켰다.

시뮬레이션 결과 오류가 발생하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-592dc825a1384b3fa46381614b6eee8\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL reset blanks digit zero time=6000
FATAL: C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\sim\tb_segment_scan8.sv:22: check failed
      Time: 6000 Scope: tb_segment_scan8.check

ERROR: 종료 코드 1: C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-592dc825a1384b3fa446381614b6eee8\simulation.log

* The terminal process "C:\Users\young\AppData\Local\Microsoft WindowsApps\python.EXE 'tools/lab1.py', 'simulate'" terminated with exit code: 1.
```

이후 오류를 수정하여 다시 시뮬레이트한 결과 복구에 성공하였다.

```
25     always @* begin
26         nibble = digits >> (index * 4);
27         select = blank ? 8'h00 : (8'h01 << index);
28
29         // abcdefg, dp; decimal point remains off.
30         case (nibble)
```

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\run-e4f47a9ba056455dbfde19e2d66b6da0\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS segment_scan8 checks=194
C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\sim\tb_segment_scan8.sv:30: $finish called at 1306000 (1ps)

시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.

WAVE C:\Users\young\Documents\fpga_lab2\circuits\08_segment_scan\build\sim\wave.vcd
```

- 오류를 분석해보면, 테스트 벤치는 그대로 유지한 상태로 코드에 오류만 추가하였기 때문에, 테스트벤치에서의 기댓값과 코드의 출력값이 일치하지 않아 오류가 발생하였다고 분석할 수 있다.
- 즉 blank=1이어도 항상 현재 자리 하나가 켜지게 만들어 오류가 발생한 것이다.

7. 증빙 및 source commit 링크(Github)

7.1. Counter

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/01_counter/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/01_counter/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/01_counter/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/01_counter/constraints/lab2_counter.xdc

7.2. Clock divider

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/02_clock_divider/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/02_clock_divider/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/02_clock_divider/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/02_clock_divider/constraints/lab2_clock_divider.xdc

7.3. register

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/03_register/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/03_register/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/03_register/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/03_register/constraints/lab2_register.xdc

7.4. Shift register

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/04_shift_register/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/04_shift_register/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/04_shift_register/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/04_shift_register/constraints/lab2_shift_register.xdc

7.5. PISO

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/05_piso/vscode/simulation.txt

VS Code waveform VCD:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/05_piso/vscode/wave.vcd

VS Code waveform image:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/05_piso/vscode/wave.png

VS Code waveform xdc:

https://github.com/dalkomys777/fpga_lab2/blob/main/circuits/05_piso/constraints/lab2_piso.xdc

7.6. Moore

VS Code simulation log:

https://github.com/dalkomys777/fpga_lab2/blob/main/evidence/06_moore/vscode/simulation.txt

VS Code waveform VCD:

[fpga_lab2/evidence/06_moore/vscode/wave.vcd at main · dalkomys777/fpga_lab2](#)

VS Code waveform image:

[fpga_lab2/evidence/06_moore/vscode/wave.png at main · dalkomys777/fpga_lab2](#)

VS Code waveform xdc:

[fpga_lab2/circuits/06_moore/constraints/lab2_moore.xdc at main · dalkomys777/fpga_lab2](#)

7.7. Mealy

VS Code simulation log:

[fpga_lab2/evidence/07_mealy/vscode/simulation.txt](#) at main · dalkomys777/fpga_lab2

VS Code waveform VCD:

[fpga_lab2/evidence/07_mealy/vscode/wave.vcd](#) at main · dalkomys777/fpga_lab2

VS Code waveform image:

[fpga_lab2/evidence/07_mealy/vscode/wave.png](#) at main · dalkomys777/fpga_lab2

VS Code waveform xdc:

[fpga_lab2/circuits/07_mealy/constraints/lab2_mealy.xdc](#) at main · dalkomys777/fpga_lab2

7.8. Segment scan

VS Code simulation log:

[fpga_lab2/evidence/08_segment_scan/vscode/simulation.txt](#) at main · [dalkomys777/fpga_lab2](#)

VS Code waveform VCD:

[fpga_lab2/evidence/08_segment_scan/vscode/wave.vcd](#) at main · [dalkomys777/fpga_lab2](#)

VS Code waveform image:

[fpga_lab2/evidence/08_segment_scan/vscode/wave.png](#) at main · [dalkomys777/fpga_lab2](#)

VS Code waveform xdc:

[fpga_lab2/circuits/08_segment_scan/constraints/lab2_segment_scan.xdc](#) at main · [dalkomys777/fpga_lab2](#)

전체 소스 커밋:

[Complete LAB2 circuits and evidence · dalkomys777/fpga_lab2@11c334e](#)

LAB_02 github 주소:

[dalkomys777/fpga_lab2](#)